

Data Engineer Coding Round

SQL + PySpark problems with solutions for the live technical round

window functions, dedup, gaps/islands, joins, incremental, Delta MERGE

The problems you'll actually get in a live technical round, with worked solutions. Try each yourself first, then check. Grouped: **SQL** (window functions, dedup, gaps/islands, growth, sessionization) and **PySpark** (clean/transform, joins, dedup, incremental).

Tip: say your approach out loud ("I'll partition by X, order by Y...") before coding — interviewers score reasoning as much as the final answer.

PART A — SQL

Assume tables: `orders(order_id, customer_id, amount, status, order_ts)`, `employees(id, name, dept, salary)`, `logins(user_id, login_date)`.

A1. 2nd highest salary per department

```
SELECT dept, name, salary FROM (  
  SELECT e.*, DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC) rk  
  FROM employees e) t  
WHERE rk = 2;
```

(DENSE_RANK handles ties; use ROW_NUMBER if you want exactly one row.)

A2. Top-3 orders per customer (top-N per group)

```
SELECT * FROM (  
  SELECT o.*, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY amount DESC) rn  
  FROM orders o) t  
WHERE rn <= 3;
```

A3. Running total of revenue per customer over time

```
SELECT customer_id, order_ts, amount,  
       SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_ts  
                          ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total  
FROM orders;
```

A4. Deduplicate — keep the latest record per order

```
SELECT order_id, customer_id, amount, status, order_ts FROM (  
  SELECT o.*, ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY order_ts DESC) rn  
  FROM orders o) t  
WHERE rn = 1;
```

(The everyday CDC/upsert pattern.)

A5. Month-over-month revenue growth (LAG)

```
WITH m AS (  
  SELECT DATE_TRUNC('month', order_ts) AS mth, SUM(amount) AS rev  
  FROM orders GROUP BY DATE_TRUNC('month', order_ts))  
SELECT mth, rev,  
  LAG(rev) OVER (ORDER BY mth) AS prev,  
  ROUND(100.0*(rev - LAG(rev) OVER (ORDER BY mth))  
    / NULLIF(LAG(rev) OVER (ORDER BY mth),0), 1) AS growth_pct  
FROM m ORDER BY mth;
```

A6. Customers with no orders (anti-join)

```
SELECT c.customer_id  
FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id  
WHERE o.customer_id IS NULL;
```

A7. Find missing dates (a login gap) — "gaps and islands" lite

```
-- days with no logins in a calendar range (using a date spine)  
SELECT d.dt  
FROM calendar d  
LEFT JOIN logins l ON l.login_date = d.dt  
WHERE d.dt BETWEEN DATE '2026-08-01' AND DATE '2026-08-31'  
  AND l.login_date IS NULL;
```

A8. Consecutive login streak per user (gaps & islands)

```
WITH g AS (  
  SELECT user_id, login_date,  
         login_date - (ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date)) AS grp  
  FROM (SELECT DISTINCT user_id, login_date FROM logins) x)  
SELECT user_id, MIN(login_date) start_date, MAX(login_date) end_date, COUNT(*) streak_len  
FROM g GROUP BY user_id, grp  
ORDER BY user_id, start_date;
```

(Trick: consecutive dates minus a row number stay constant, so they group together.)

A9. Sessionize events (30-min inactivity gap = new session)

```
WITH e AS (  
  SELECT user_id, event_ts,  
         CASE WHEN event_ts - LAG(event_ts) OVER (PARTITION BY user_id ORDER BY event_ts)  
               > INTERVAL '30' MINUTE OR LAG(event_ts) OVER (PARTITION BY user_id ORDER BY  
event_ts) IS NULL  
         THEN 1 ELSE 0 END AS new_session  
  FROM events)  
SELECT user_id, event_ts,  
       SUM(new_session) OVER (PARTITION BY user_id ORDER BY event_ts) AS session_id  
FROM e;
```

A10. Pivot: revenue by status per customer (conditional aggregation)

```
SELECT customer_id,  
       SUM(CASE WHEN status='completed' THEN amount ELSE 0 END) AS completed_rev,  
       SUM(CASE WHEN status='cancelled' THEN amount ELSE 0 END) AS cancelled_rev  
FROM orders GROUP BY customer_id;
```

PART B — PySpark

Setup: `from pyspark.sql import functions as F, Window`

B1. Clean & standardize raw orders → partitioned Parquet

```
clean = (orders
  .filter((F.col("status") != "cancelled") & (F.col("amount") > 0))
  .withColumn("amount", F.col("amount").cast("double"))
  .withColumn("order_date", F.to_date("order_ts"))
  .dropDuplicates(["order_id"]))
clean.write.mode("overwrite").partitionBy("order_date").parquet("s3://.../curated/orders/")
```

B2. Top-N per group (window)

```
w = Window.partitionBy("customer_id").orderBy(F.col("amount").desc())
top3 = orders.withColumn("rn", F.row_number().over(w)).filter("rn <= 3").drop("rn")
```

B3. Deduplicate keeping the latest (CDC pattern)

```
w = Window.partitionBy("order_id").orderBy(F.col("order_ts").desc())
latest = orders.withColumn("rn", F.row_number().over(w)).filter("rn = 1").drop("rn")
```

B4. Revenue by region with a broadcast join

```
rev = (orders.filter("status = 'completed'")
  .join(F.broadcast(regions), "region_id")
  .groupBy("region_name")
  .agg(F.round(F.sum("amount"), 2).alias("revenue"))
  .orderBy(F.col("revenue").desc()))
```

B5. Running total (window with frame)

```
w = (Window.partitionBy("customer_id").orderBy("order_ts")
  .rowsBetween(Window.unboundedPreceding, Window.currentRow))
orders.withColumn("running_total", F.sum("amount").over(w))
```

B6. Explode nested JSON (array → rows)

```
raw = spark.read.option("multiline", True).json("s3://.../events.json")
items = (raw.select(F.explode("items").alias("i"))
  .select(F.col("i.sku").alias("sku"),
    F.col("i.qty").cast("int").alias("qty")))
```

B7. Incremental load (watermark) — only new rows

```
last_ts = spark.read.parquet("s3://.../curated/orders/").agg(F.max("order_ts")).first()[0]
new = orders.filter(F.col("order_ts") > F.lit(last_ts))
new.write.mode("append").partitionBy("order_date").parquet("s3://.../curated/orders/")
```

B8. Upsert into a Delta table (MERGE) — SCD Type 1

```
from delta.tables import DeltaTable
tgt = DeltaTable.forPath(spark, "s3://.../gold/customers")
(tgt.alias("t").merge(updates.alias("s"), "t.id = s.id")
 .whenMatchedUpdateAll()
 .whenNotMatchedInsertAll()
 .execute())
```

B9. Handle nulls & bad data

```
df.na.drop(subset=["order_id"]) # drop rows missing the key
df.na.fill({"amount": 0, "status": "unknown"}) # fill defaults
df.withColumn("amount", F.when(F.col("amount") < 0, 0).otherwise(F.col("amount")))
```

B10. Word/event count (classic)

```
(spark.read.text("s3://.../logs/")
 .select(F.explode(F.split("value", r"\s+")).alias("word"))
 .groupBy("word").count()
 .orderBy(F.col("count").desc()).show())
```

B11. Read a plan / spot a shuffle

```
rev.explain(True) # look for 'Exchange' (shuffle) and BroadcastHashJoin vs SortMergeJoin
```

What interviewers look for (say these while coding)

- **State the approach first** — "partition by key, order by timestamp, keep row_number = 1."
- **Handle edge cases** — nulls, duplicates, ties, empty input.
- **Think performance** — broadcast small tables, filter early, avoid UDFs, mind the shuffle.
- **Make it idempotent** — overwrite-by-partition or MERGE so re-runs are safe.
- **Write readable code** — clear names, small steps, comments where non-obvious.

Practice setup (free)

- **SQL:** run in DuckDB, SQLite, or Postgres locally.
- **PySpark:** `pip install pyspark` locally, or a **Databricks Community Edition** notebook. Do 5–6 of each out loud before the interview.

